

ATTORNEY SHIELD

ASI 2.0 - Native Android & iOS

ASI 2.0 Development Plan

Native Android & iOS member apps

Draft for approval - 2026-08-10

Colour system: Attorney-Shield Color Scheme Review

Repository: samson-phillip/asi-claude

DRAFT FOR APPROVAL

Contents

1. Objective	3
2. Reference authority	3
3. Scope reality check - read this before estimating	3
What we can build today	4
4. Target architecture	4
Android (kotlin)	4
iOS (swift)	4
Design system as code	5
5. Behaviour to port verbatim	5
6. Phases	5
7. Cross-cutting requirements	6
8. Risks	6
9. Open questions - need answers before Phase 3	7
10. Immediate next steps	7

Date: 2026-08-10 **Scope:** Native Android (kotlin) and iOS (swift) member apps **Status:** Draft for approval - no code written yet

1. Objective

Ship native Android and iOS member apps that match member-client's behaviour and API usage, rendered in the Attorney Shield visual system from the colour PDF and the CodePen onboarding reference.

2. Reference authority

Rank	Source	Governs
1	design/color-system.md (from the colour PDF)	All colour values
2	CodePen "Attorney - Shield App V6"	Layout, typography, screen inventory, flow
3	member-client	Behaviour, API contracts, state machines, errors

Full rationale and the resolved PDF-vs-CodePen conflicts are in [design/color-system.md](#). Two rules that will otherwise bite: member-client's **blue/violet CSS is not the palette**, and **never white text on gold** (3.13:1, fails WCAG AA).

3. Scope reality check - read this before estimating

The two references describe **very different product sizes**, and the API only supports the smaller one.

CodePen describes: 35 screens, 5 stages, 3 user types (Standard Member, 7-Day Limited Trial, Guest), an app->web handoff, a paywall with pricing (\$19/mo, \$179/yr, \$29/mo family), and an in-app nudge system.

member-client **implements:** 3 screens - LoginScreen -> HomeScreen -> CallScreen.

The documented API supports only that smaller set. Every operation in member-client/src/lib/api.ts:

- GraphQL POST /query - login, user, casesByUser, adminIncidentTypeList + adminLanguageList, partnerAttorneys
- REST POST {API_BASE_URL}/api/vonage/video/member-call

There is **no endpoint for sign-up, payment, subscription, trial activation, or guest sessions**. CLAUDE.md forbids inventing endpoints. Therefore:

Stages covering account creation, payment, trial, and guest access are BLOCKED on backend work that does not exist yet. They cannot be built as functioning features, only as static UI against mocks.

This is the single biggest planning risk and needs a decision before Phase 3 (\$6). Options in \$9.

What we can build today

Capability	API support	Buildable now
Email/password login	login	Yes
Member profile / org	user	Yes
Case -> jurisdiction/partner/case resolution	casesByUser	Yes
Incident-type tiles (multilingual)	adminIncidentTypeList + adminLanguageList	Yes
Attorney pre-selection	partnerAttorneys	Yes
Place video call + Vonage credentials	member-call	Yes
Welcome/onboarding carousel	none needed (static)	Yes
Sign-up, payment, trial, guest	none	No Blocked
Token refresh	none used	No See §8

4. Target architecture

Mirror structure across platforms so one review covers both.

```
<repo>/
core/design/ tokens (colour, type, spacing), reusable components
core/network/ GraphQL client, REST client, error mapping
core/session/ auth + resolved member context, persistence
feature/auth/ login
feature/home/ greeting, attorney chips, incident tiles
feature/call/ Vonage video, phase machine
```

Android (kotlin)

- Jetpack Compose, Material 3 with a fully overridden colour scheme
- MVVM, StateFlow for state; Hilt for DI
- Ktor or OkHttp + kotlinx.serialization. **Raw** POST /query, not Apollo - faithfulness to api.ts's hand-written queries matters more than codegen, and it avoids requiring a schema we do not have
- Vonage Video Android SDK
- EncryptedSharedPreferences for the session

iOS (swift)

- SwiftUI, @Observable view models
- URLSession + Codable, same raw-GraphQL approach
- Vonage Video iOS SDK
- Keychain for the session

Design system as code

Ship design/color-system.md \$6 tokens as a single source file per platform (AsiColors.kt, AsiColors.swift). **No raw hex literals in UI code** - this is a lint/review gate, not a convention (§ testing plan).

5. Behaviour to port verbatim

From member-client. These are the details that make the app correct:

- 1 **Call routing** - send attorneyId when the member pre-selected one, **else** queueId. Never both.
- 2 **Call phase machine** - exactly starting -> connecting -> live -> ended, plus no-attorney and error terminals.
- 3 409 **on** member-call = NoAttorneyError, a *retryable* message. Never a crash, never a generic failure.
- 4 **Incident labels are multilingual** - English -> org default language -> first available -> humanized code. Sort by sortOrder.
- 5 **Location is best-effort** - never block a call on geolocation. Resolve null on unsupported/denied/timeout.
- 6 listAttorneys **never throws** - empty list means auto-match/queue only, and the chip row hides entirely.
- 7 **Login never blocks on reference lookups** - user and casesByUser failures fall back to DEV_DEFAULTS.
- 8 **Validate video credentials** - reject a member-call response missing apiKey/sessionId/token/callId/videoRoomId.
- 9 **Publisher/session teardown** on unmount - unpublish, destroy, disconnect, each independently guarded.

6. Phases

Each phase ends with: unit tests green, emulator + simulator verified against member-client in Claude Browser, and a journal entry.

Phase 0 - Foundations (*no product surface*) Scaffold both projects with test harnesses. Encode design tokens and verify the contrast table programmatically. Establish CI-runnable test commands. *Exit*: ./gradlew test and xcodebuild test both run green on a trivial test; token contrast test passes.

Phase 1 - Network + session GraphQL and REST clients, error mapping, session persistence, member-context resolution with DEV_DEFAULTS fallback. *Exit*: login against dev returns a token and resolves org/jurisdiction; unit tests cover the fallback paths in §5.7.

Phase 2 - Login -> Home -> Call (the working spine) The three member-client screens in the Attorney Shield visual system. Includes the welcome/onboarding carousel (static, no API needed). *Exit*: a real video call connects on both platforms; all six call phases reachable in tests; behaviour parity checklist (§5) signed off.

Phase 3 - Onboarding journey (*gated* - see §3) Remaining CodePen stages. **Requires a decision on the missing endpoints first.** Built against mocks only if we explicitly choose the mock path.

Phase 4 - Hardening Accessibility audit, dynamic type, rotation, offline/airplane-mode, token expiry, permission-denied paths, performance.

Phases 0-2 are the committed plan. Phase 3 is deliberately unestimated until §9 is answered.

7. Cross-cutting requirements

- **Accessibility:** WCAG AA (4.5:1 text / 3:1 UI) enforced by test, not review. Respect OS font scaling and screen readers; every icon-only control labelled.
- **Permissions:** camera and microphone are required for the call; location is optional and must degrade silently. Request at point of use with a clear rationale, and handle permanent denial.
- **No secrets in the repo.** `apiKey` arrives per-call from the backend - never bundle Vonage credentials.
- **Config per environment**, mirroring `config.ts` (`GRAPHQL_URL`, `API_BASE_URL`, `DEV_DEFAULTS`, `CALL_TTL_SECONDS = 3600`).

8. Risks

Risk	Impact	Mitigation
Missing endpoints for signup/payment/trial/guest	Blocks ~2 of 5 stages	Resolve §9 before Phase 3; Phases 0-2 are unaffected
No token refresh exists - <code>login</code> returns a <code>refreshToken</code> but <code>api.ts</code> never sends it; there is no refresh operation	Sessions die with no recovery path; native apps are backgrounded far longer than a web tab, so this hurts more than it does on web	Spike in Phase 1. Interim: detect auth failure and re-prompt login cleanly
Web->native video SDK gap - reference uses <code>@vonage/client-sdk-video</code> ; native needs the Vonage Video Android/iOS SDKs	Could invalidate Phase 2 estimates	De-risk in Phase 0 with a spike proving <code>apiKey/sessionId/token</code> connect natively
<code>member-call</code> has no auth today	Abuse surface	Flag to backend; do not design around it
CodePen is v11 / "locked reference" but PDF conflicts with it	Rework	Authority order in §2; conflicts already resolved
409 handling regressions	Members see a crash at the worst moment	Explicit test case, both platforms
Reference app not yet run locally	Parity checks can't start	Stand it up in Phase 0 (needs <code>serve-proxy.mjs</code>)

9. Open questions - need answers before Phase 3

- 1 **Sign-up / payment / trial / guest endpoints - do they exist?** If not, is the intent that (a) backend builds them, (b) we build static UI against mocks, or (c) onboarding stays on web and the app deep-links into it? The CodePen's "App <-> Web handoff" badge hints at (c), which would substantially reduce native scope. **This is the highest-value question to settle.**
- 2 **Token refresh** - is there a refresh operation on the gateway we should use?
- 3 **Error/danger colour** - the palette forbids #FF4500 and supplies no error colour. What should errors and hang-up use? (See colour-system §6.)
- 4 **Payments** - App Store / Play billing, or web checkout? This decides whether store review is on the critical path.
- 5 **Which CodePen palette values are stale** - confirm the PDF supersedes Deep Navy #0A1626, Live Green #2E9E5B, Trust Steel #1A5FA8.
- 6 **Guest user type** - what can a guest actually do without an account?

10. Immediate next steps

- 1 Get §9.1 and §9.3 answered.
- 2 Phase 0: scaffold both repos, encode tokens, **run the Vonage native spike.**
- 3 Stand up `member-client` locally for parity comparison.